Biological Sequences

BLAST: similarity search

Dario Pescini

Università degli Studi di Milano-Bicocca

Dipartimento di Statistica e Metodi Quantitativi

dario.pescini@unimib.it

BLAST — Basic Local Alignment Search Tool

Definition

BLAST is the standard tool for searching similar sequences in large biological databases via heuristic local alignment.

Original paper: Altschul et al., 1990 (*doi: 10.1016/S0022-2836(05)80360-2*)

Program	Query vs DB
`blastn`	DNA vs DNA
`blastp`	Protein vs protein
`blastx`	Translated DNA vs protein
`tblastn`	Protein vs translated DNA
`tblastx`	Translated DNA vs translated DNA

The BLAST algorithm — overview

1. fragment the query sequence, in all the possible one character overlapping words of length `W`
   `'TDTLSH', 'W = 3 -> ['TDT', 'DTL', 'TLS', 'LSH']`
2. for each obtained word, generate all the possible affine words `W-mer` exploiting a substitution matrix (e.g. BLOSUM) and associate them a score obtained by an alignment without gaps.
   `'LSH', W = 3 -> [('LSH', 16), ('ISH', 14), ('MSH', 14), ('VSH', 13), ...]`
3. select all and only the `W-mer` whose score is `greater than` a fixed threshold `T`
   `[('LSH', 16), ('ISH', 14), ('MSH', 14)]`

The BLAST algorithm — overview

1. select in the Database all and only the sequences containg at least one fragment of lenght W identical to one of the W-mer. One of these sequences is called `hit`
2. extend the W-mer in each hit in the forward and back direction (alignment with no gaps). Extending the seeds.
3. halt the extension process of each seed if the score falls below a fixed threshold `X`. Generation of an High-scoring Segment Pair (HSP)
4. select all and only those HSPs whose score is greater than a fixed threshold `S`

Key BLAST parameters

Parameter	Meaning
W	*Word size*: length of the seed word; default 3 (protein) or 11 (DNA)
T	Score threshold for *W*-mers to be used as seeds
X	*Drop-off*: extension stops if the score drops by *X* from the maximum
E-value	Expected number of random hits with score $\geq S$; E-value < 0.001 is conventionally significant

E-value vs p-value

The E-value depends on the database size: the same alignment will yield a higher E-value (less significant) against a larger database.

Local vs remote BLAST

Local BLAST

- Faster
- Allows custom databases
- Requires BLAST+ installed

```
blastx -query seq.fasta
-db nr -out res.xml
-evalue 0.001 -outfmt 5
```

Remote BLAST (NCBI)

- No installation required
- Direct access to `nr`, `nt`, `refseq_protein`, ...
- Slower; limited to public DBs
- Via Biopython: `NCBIWWW.qblast`

Remote BLAST — qblast

```
from Bio.Blast import NCBIWWW
from Bio import SeqIO

record = SeqIO.read(
    'data/DRR076693.3.fasta', 'fasta')
print(record.seq)

blast_result = NCBIWWW.qblast(
    'blastx',             # programma
    'refseq_protein',    # database
    record.format('fasta'))  # sequenza query
```

Parameter	Common values
program	'blastn','blastp','blastx',…
database	'nt','nr','refseq_protein',…
expect	E-value threshold (default 10)
format_type	'XML','HTML','Text'

Saving the XML result

The result of `qblast` is a file handle: it must be read and saved before closing it.

```
with open(
    'data/blastOutput_DRR076693.3.xml', 'w') as f:
    f.write(blast_result.read())
blast_result.close()
```

Warning

`qblast` can take several minutes for long sequences or large databases. Always save the XML to avoid repeating the search.

Parsing with NCBIXML

```
from Bio.Blast import NCBIXML

result_handle = open(
    'data/blastOutput_DRR076693.3.xml', 'r')
blast_record = NCBIXML.read(result_handle)

print('Query:', blast_record.query)
print('Numero di hit:',
      len(blast_record.alignments))
```

- `NCBIXML.read(handle)` — reads a single record (one query)
- `NCBIXML.parse(handle)` — returns an *iterator* for multiple results (multiple queries in the same file)
- Each record has: `.query`, `.alignments`, `.descriptions`

Extracting information from hits

```
for hit in blast_record.alignments:
    print('Accession:', hit.accession)
    print('Descrizione:', hit.hit_def)
    print('Lunghezza:', hit.length)
    for hsp in hit.hsps:
        print('  E-value:', hsp.expect)
        print('  Score:', hsp.score)
        print('  Identita:', hsp.identities)
```

- Each `alignment` can have one or more `hsp` (*High-scoring Segment Pair*)
- The first HSP (`hit.hsps[0]`) is generally the best
- `hsp.identities` = number of identical positions; `hsp.positives` = conservative positions

Retrieving the organism name

We use `Entrez.efetch` to enrich the results with the organism name:

```
from Bio import Entrez
Entrez.email = 'nome@esempio.it'

for hit in blast_record.alignments:
    ncbi_id = hit.accession
    h = Entrez.efetch(
        db='protein', id=ncbi_id,
        rettype='gp', retmode='xml')
    rec = Entrez.read(h)
    organism = rec[0][0]['GBSeq_organism']
    print(ncbi_id, '->', organism)
```

- `rettype='gp'` returns the GenBank/GenPept format
- `GBSeq_organism` contains the scientific name of the organism

Collecting results into a DataFrame

```
import pandas as pd

rows = []
for hit in blast_record.alignments:
    rows.append({
        'ncbi_id':     hit.accession,
        'description': hit.hit_def,
        'length':      hit.length,
        'evalue':      hit.hsps[0].expect,
    })

df = pd.DataFrame(rows)
df.to_excel('data/blast_results.xlsx',
            index=False)
print(df.head())
```

- Building a dictionary for each hit and collecting them in a list is the most efficient pattern for creating DataFrames from a loop
- `df.to_excel` requires `openpyxl` (`conda install openpyxl`)

Complete workflow

1. 1. Read the FASTA file with `SeqIO.read`
2. 2. Run remote BLAST with `NCBIWWW.qblast` specifying program, database, sequence, and E-value
3. 3. Save the XML output to disk (avoid repeating the search)
4. 4. Parse the result with `NCBIXML.read`
5. 5. Iterate over `alignments`: extract accession, description, length, E-value
6. 6. Enrich with the organism name via `Entrez.efetch`
7. 7. Collect into a `DataFrame` and save to Excel with `pd.to_excel`